ADA BOOST

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

[2]:
```
dataset = pd.read_csv('insurance.csv')
dataset
```

[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

1338 rows × 7 columns

[3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

[3]:

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	False	True	False	False	True
1	18	33.770	1	1725.55230	True	False	False	True	False
2	28	33.000	3	4449.46200	True	False	False	True	False
3	33	22.705	0	21984.47061	True	False	True	False	False
4	32	28.880	0	3866.85520	True	False	True	False	False
...	...	...	...	...	...	...	...	...	...
1333	50	30.970	3	10600.54830	True	False	True	False	False
1334	18	31.920	0	2205.98080	False	False	False	False	False
1335	18	36.850	0	1629.83350	False	False	False	True	False
1336	21	25.800	0	2007.94500	False	False	False	False	True
1337	61	29.070	0	29141.36030	False	True	True	False	False

1338 rows × 9 columns

[4]:
```
indep=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
dep=dataset['charges']
```

[5]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

[6]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

[18]:
```
from sklearn.ensemble import AdaBoostRegressor
# regr = AdaBoostRegressor(random_state=0, n_estimators=100)
# regr.fit(X, y)
from sklearn.model_selection import GridSearchCV
param_grid = {'estimator':[None,'object'],
              'learning_rate': [1.0, 1.5, 2.0,2.5],
'random_state': [None, 0, 10, 100, 1000]}
```

[19]:
```
grid = GridSearchCV(AdaBoostRegressor(), param_grid, refit = True, verbose=3,n_jobs=-1)
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 40 candidates, totalling 200 fits
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:489: FitFailedWarning:
100 fits failed out of a total of 200.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
100 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 851, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\base.py", line 1393, in wrapper
    estimator._validate_params()
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\base.py", line 554, in _validate_params
    validate_parameter_constraints(
```

[19]:

```
GridSearchCV
best_estimator_: AdaBoostRegressor
AdaBoostRegressor
```

[20]:

```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'estimator': None, 'learning_rate': 1.5, 'random_state': 10}:
```

[21]:

```
table=pd.DataFrame.from_dict(re)
table
```

[21]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_estimator	param_learning_rate	param_random_state	params	split0_test_score	sp
0	0.224161	0.020407	0.014243	0.002104	NaN	1.0	None	{'estimator': None, 'learning_rate': 1.0, 'ran...	0.750347	
1	0.178135	0.047157	0.012937	0.003001	NaN	1.0	0	{'estimator': None, 'learning_rate': 1.0, 'ran...	0.782168	
2	0.114914	0.022890	0.013518	0.003441	NaN	1.0	10	{'estimator': None, 'learning_rate': 1.0, 'ran...	0.777396	

XG_BOOST

[1]:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
dataset = pd.read_csv('insurance.csv')
dataset
```

[1]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

1338 rows × 7 columns

[2]:

```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

[2]:

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest

[2]:

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	False	True	False	False	True
1	18	33.770	1	1725.55230	True	False	False	True	False
2	28	33.000	3	4449.46200	True	False	False	True	False
3	33	22.705	0	21984.47061	True	False	True	False	False
4	32	28.880	0	3866.85520	True	False	True	False	False
...	...	...	...	...	...	...	...	...	...
1333	50	30.970	3	10600.54830	True	False	True	False	False
1334	18	31.920	0	2205.98080	False	False	False	False	False
1335	18	36.850	0	1629.83350	False	False	False	True	False
1336	21	25.800	0	2007.94500	False	False	False	False	True
1337	61	29.070	0	29141.36030	False	True	True	False	False

1338 rows × 9 columns

[3]:
```
indep=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
dep=dataset['charges']
```

[4]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

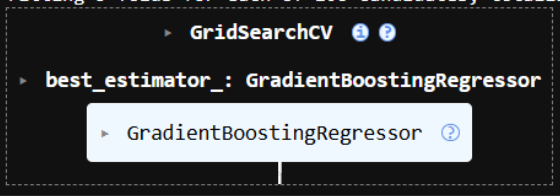
[5]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

[8]:
```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 150],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [2, 3, 4],
    'subsample': [0.8, 1.0],
    'random_state': [0, 10]
}
```

[9]:
```
grid = GridSearchCV(GradientBoostingRegressor(), param_grid, refit = True, verbose=3,n_jobs=-1)
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 108 candidates, totalling 540 fits
```

[9]:



[10]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'learning_rate': 0.05, 'max_depth': 2, 'n_estimators': 150, 'random_state': 10, 'subsample': 0.8}:
```

[11]:
```
table=pd.DataFrame.from_dict(re)
table
```

[11]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_learning_rate	param_max_depth	param_n_estimators	param_random_state	param_subsa
0	0.258141	0.006816	0.005692	0.000894	0.01	2	50	0	
1	0.202776	0.032461	0.004904	0.000258	0.01	2	50	0	
2	0.193217	0.013088	0.004810	0.000439	0.01	2	50	10	
3	0.175573	0.013823	0.004362	0.000803	0.01	2	50	10	

LG_Boost

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
dataset = pd.read_csv('insurance.csv')
#dataset
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

[1]:

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	False	True	False	False	True
1	18	33.770	1	1725.55230	True	False	False	True	False
2	28	33.000	3	4449.46200	True	False	False	True	False
3	33	22.705	0	21984.47061	True	False	True	False	False
4	32	28.880	0	3866.85520	True	False	True	False	False
...	...	...	...	...	...	...	...	...	...
1333	50	30.970	3	10600.54830	True	False	True	False	False
1334	18	31.920	0	2205.98080	False	False	False	False	False
1335	18	36.850	0	1629.83350	False	False	False	True	False
1336	21	25.800	0	2007.94500	False	False	False	False	True
1337	61	29.070	0	29141.36030	False	True	True	False	False

1338 rows × 9 columns

[2]:
```
indep=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
dep=dataset['charges']
```

[3]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

[4]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

[5]:
```
%pip install lightgbm
```

```
Requirement already satisfied: lightgbm in C:\Users\Admin\anaconda3\Lib\site-packages (4.7.0)
Requirement already satisfied: narwhals>=1.15 in C:\Users\Admin\anaconda3\Lib\site-packages (from lightgbm) (2.23.0)
Requirement already satisfied: numpy>=1.21.3 in C:\Users\Admin\anaconda3\Lib\site-packages (from lightgbm) (2.4.6)
Requirement already satisfied: scipy in C:\Users\Admin\anaconda3\Lib\site-packages (from lightgbm) (1.18.0)
Note: you may need to restart the kernel to use updated packages.
```

[6]:
```
from lightgbm import LGBMRegressor
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 150],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [-1, 3, 5],
    'num_leaves': [15, 31, 50],
    'subsample': [0.8, 1.0],
    'random_state': [0, 10]
}
```

[7]:
```
grid = GridSearchCV(LGBMRegressor(), param_grid, refit = True, verbose=3,n_jobs=-1)
grid.fit(X_train, y_train)
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

[7]:

GridSearchCV

best_estimator_: LGBMRegressor

LGBMRegressor

[8]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100, 'num_leaves': 15, 'random_state': 0, 'subsample':
0.8}:
```

[9]:
```
table=pd.DataFrame.from_dict(re)
table
```

[9]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_learning_rate	param_max_depth	param_n_estimators	param_num_leaves	param_random_
0	4.709779	0.143589	0.010764	0.001357	0.01	-1	50	15	